Promises

9 min read Updated 24 Jul 2026 Topic 17 of 70

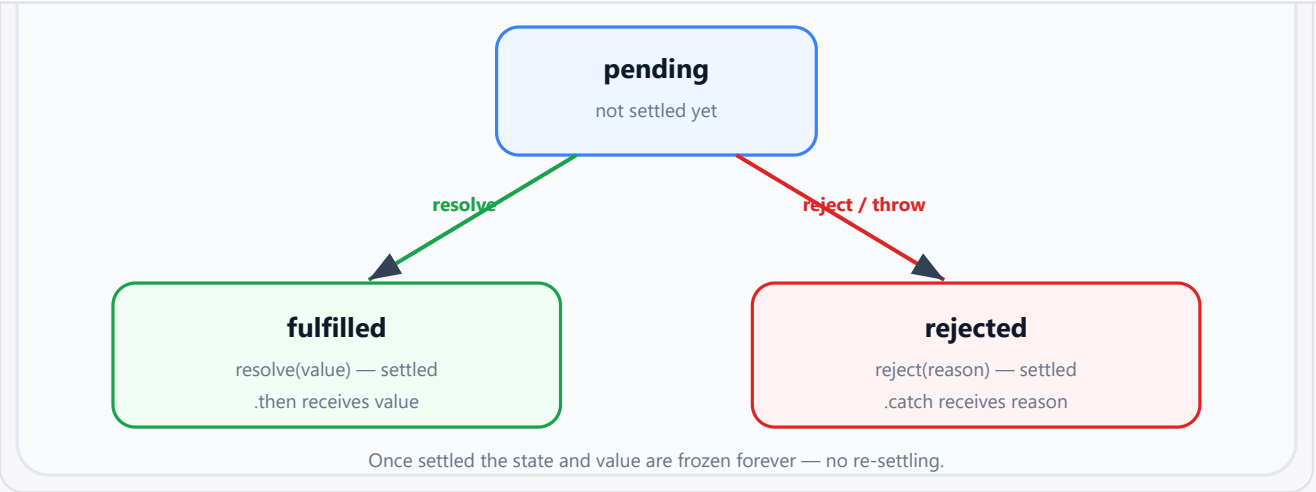
Why it matters

Promises are the backbone of modern asynchronous JavaScript. `async/await` — the syntax you probably reach for first — is not a separate mechanism; it is sugar over promises. Every `await` unwraps a promise, and every `async` function returns one. If your mental model of promises is fuzzy, then `async/await` will occasionally do surprising things and you won't know why. Master promises and the rest of asynchronous Node stops being mysterious.

Promises also solved the concrete problems callbacks created: the rightward nesting, the duplicated error checks, the inability to compose. A promise is a *value you can pass around and combine* — you can return it, store it, hand it to `Promise.all`, or chain transformations onto it. That composability is why the whole ecosystem migrated. In interviews, promise combinators (`all`, `allSettled`, `race`, `any`) and the exact rules of chaining are among the most common questions because they reveal whether you can reason about concurrency, not just await one thing.

The concept, simply

A promise is an object representing the eventual result of an asynchronous operation. It is a placeholder for a value that isn't ready yet. A promise is always in exactly one of three states: **pending** (still working), **fulfilled** (finished with a value), or **rejected** (failed with a reason). The move from pending to either fulfilled or rejected is called **settling**, and it happens *at most once*. After a promise settles, its state and value are frozen forever — you cannot un-settle it or change the value.



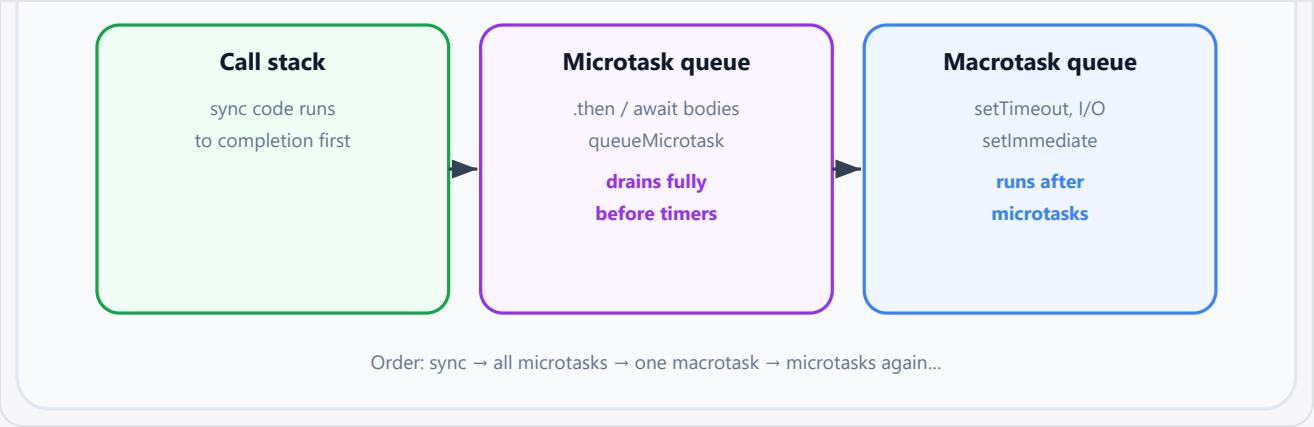
A promise settles once and only once; the transition is one-way and irreversible.

You read a promise's outcome with three methods. `.then(onFulfilled, onRejected)` registers what to do when it fulfils (and optionally when it rejects). `.catch(onRejected)` handles rejection. `.finally(fn)` runs a cleanup callback regardless of outcome, without touching the value. All three return *new* promises, which is what makes chaining possible.

How it works

Consider the resolution process. When you construct `new Promise((resolve, reject) => { ... })`, the executor runs **synchronously** right away. Calling `resolve(value)` settles the promise as fulfilled; `reject(reason)` settles it as rejected. Crucially, resolving with another promise (a "thenable") doesn't fulfil with that promise as a value — it *adopts* it, waiting for the inner promise to settle first. This flattening is why you rarely see promises nested inside promises.

The handlers you attach with `.then` never run synchronously, even if the promise is already settled. They are scheduled as **microtasks**. The microtask queue drains completely after the current synchronous call stack finishes and *before* the event loop processes the next macrotask (like a `setTimeout` callback or I/O event). This is why a `.then` callback runs before a `setTimeout(fn, 0)` queued at the same time.



Promise callbacks are microtasks: they drain fully before the next timer or I/O callback.

Chaining works because each `.then` returns a new promise resolved with whatever its callback returns. Return a plain value and the next `.then` receives it; return a promise and the chain waits for it to settle before continuing. **Throw** inside a handler (or return a rejected promise) and the chain skips every subsequent `.then` until it hits a `.catch` — errors propagate down the chain looking for a rejection handler, exactly like exceptions bubble up a call stack.

Two mechanics are worth pinning down because interview answers hinge on them. First, `Promise.resolve(x)` returns `x` unchanged when it is already a native promise and otherwise wraps it — so it is the cheap, correct way to normalise a "value or promise" without the `new Promise` anti-pattern. Second, a "thenable" is *any* object with a `.then` method, not only a real `Promise`; the resolution algorithm calls that `.then` to adopt it, which is how promises from different libraries interoperate. Adopting a thenable costs an **extra microtask tick** — the engine schedules an internal `.then` to unwrap it — so a handler that returns `Promise.resolve(v)` resumes one tick later than one that returns `v` directly. This rarely affects correctness but explains surprising ordering in tight benchmarks.

Working through an explicit ordering puzzle cements the microtask rule. Predict the output before reading it:

JAVASCRIPT

Copy

```
console.log('1 sync');
setTimeout(() => console.log('2 timeout'), 0);
```

```
// Output:
// 1 sync          ← runs now, on the call stack
// 5 sync          ← runs now, on the call stack
// 3 microtask     ← microtask queue drains before any timer
// 4 microtask     ← same queue, in registration order
// 2 timeout       ← macrotask — only after every microtask
```

A single table captures how the four combinators differ on the two axes that matter — when they settle and how they treat failure:

Combinator	Fulfils when	Rejects when
`Promise.all`	Every input fulfils	First input rejects (fail-fast)
`Promise.allSettled`	All inputs settle	Never
`Promise.race`	First input to settle was a fulfilment	First input to settle was a reje
`Promise.any`	First input fulfils	Every input rejects

All four accept any iterable and, apart from `allSettled`, propagate the first relevant settlement without cancelling the others — there is no cancellation primitive in the promise model, so siblings always run to completion regardless of which combinator you chose.

Code examples

Promisifying a callback API by hand. The executor runs now; `resolve` / `reject` settle it later from inside the callback:

JAVASCRIPT Copy

```
function delay(ms, value) {
  return new Promise((resolve, reject) => {
```

```
}

delay(100, 'done').then((v) => console.log(v)); // 'done' after 100ms
```

A chain that transforms a value step by step. Each `.then` hands its return to the next; the single `.catch` at the end handles a failure anywhere above it:

JAVASCRIPT Copy

```
fetchUser(1)
  .then((user) => fetchOrders(user.id)) // return a promise → chain waits
  .then((orders) => orders.filter((o) => o.paid))
  .then((paid) => console.log('paid orders:', paid.length))
  .catch((err) => console.error('chain failed:', err.message))
  .finally(() => console.log('done, success or not'));
```

The four combinators cover distinct concurrency needs. `Promise.all` fails fast; `allSettled` reports every outcome:

JAVASCRIPT Copy

```
// all: need every result; one rejection rejects the whole thing.
const [a, b, c] = await Promise.all([load('a'), load('b'), load('c')]);

// allSettled: never rejects — inspect each result individually.
const results = await Promise.allSettled([load('a'), load('b')]);
for (const r of results) {
  if (r.status === 'fulfilled') console.log('ok', r.value);
  else console.log('failed', r.reason.message);
}
```

`Promise.race` settles on the first to finish (win or lose) — ideal for timeouts. `Promise.any` waits for the first *success*, ignoring rejections until all fail:

JAVASCRIPT Copy

```
// ... whichever settles first wins.
function withTimeout(promise, ms) {
  const timeout = new Promise((_, reject) =>
    setTimeout(() => reject(new Error('timed out')), ms));
  return Promise.race([promise, timeout]);
}

// any: first mirror to succeed; rejects with AggregateError only if all f
const fastest = await Promise.any([mirror1(), mirror2(), mirror3()]);
```



Pick the combinator by intent: every result (all), every outcome (allSettled), first settle (race), first success (any).

Common mistakes & gotchas

- **Forgetting to return inside `.then`.** If you call `fetchOrders( ... )` without `return`, the chain doesn't wait for it and the next `.then` gets `undefined`. Always return the promise you want the chain to await.
- **The floating promise.** Starting a promise without `await`, `return`, or `.catch` means its rejection goes unhandled and its completion is unordered relative to your code.
- **The forgotten `.catch`.** An unhandled rejection triggers a process warning and, by default in current Node, can crash the process. Every chain needs a terminal

- **The explicit-construction anti-pattern.** Wrapping a function that already returns a promise in `new Promise((resolve) ⇒ fn().then(resolve))` loses error propagation and is pointless — just return `fn()`.
- **Assuming `Promise.all` cancels siblings on rejection.** It rejects immediately, but the other promises keep running to completion — there is no cancellation. Their later rejections may become unhandled.
- **Mixing up `race` and `any`.** `race` settles on the first to settle, so an early rejection wins; `any` ignores rejections and waits for the first fulfilment.
- **Expecting a settled promise's `.then` to run synchronously.** It never does — it's always a microtask, so ordering with sync code can surprise you.

Best practices

- Always terminate a chain with `.catch`, or `await` it inside `try/catch` — never leave a rejection unhandled.
- Return the promise from inside `.then` so the chain waits; a missing return is the most common promise bug.
- Prefer the built-in combinators over hand-rolled counters, and choose the one that matches your intent (all / allSettled / race / any).
- Reject with `Error` objects, not strings, so you keep a stack trace and a `.message`.
- Don't wrap existing promises in `new Promise`; return or chain them directly.
- Use `allSettled` when partial failure is acceptable and you need to report each outcome; use `all` only when you truly need every result.

⭐ Interview lens

> **Q: What are the three states of a promise and which transitions are legal?**
> Pending, fulfilled, rejected. It starts pending and moves once — pending → fulfilled or pending → rejected. Settling is one-way and happens at most once; the value is then frozen.

No. `.then` callbacks are always scheduled as microtasks, running after the current synchronous stack finishes but before the next timer or I/O callback.

Q: Difference between `Promise.all` and `Promise.allSettled`?
`all` rejects as soon as any input rejects (fail-fast) and otherwise fulfils with an array of values. `allSettled` never rejects; it waits for all and returns an array of `{status, value|reason}` descriptors.

Q: `Promise.race` vs `Promise.any`? (trap)
`race` settles with the first promise to *settle*, so a fast rejection can win. `any` settles with the first to *fulfil*, ignoring rejections, and only rejects (with an `AggregateError`) if every input rejects.

Q: If `Promise.all` rejects, what happens to the other promises?
Nothing stops them — there's no cancellation. They keep running; `all` just ignores their later results. If one of those later rejects with no handler, you can get an unhandled rejection.

Q: What does `resolve`-ing a promise with another promise do?
It adopts the inner promise: the outer stays pending until the inner settles, then takes on its state and value. This thenable-flattening is why promises don't nest.

Cheat-card

- **States:** pending → fulfilled | rejected. Settles once, frozen after.
- **Read with:** `.then` / `.catch` / `.finally` — each returns a new promise.
- **Handlers:** always microtasks; run before timers, never sync.
- **all** = every result, fail-fast. **allSettled** = every outcome, never rejects.

QUICK CHECK — no marks, just for you.

1. What are the three states of a promise, and how many times can it settle?

- [ ] pending, fulfilled, rejected — and it settles at most once
- [ ] open, closed, error — settles any number of times
- [ ] waiting, done, failed — settles exactly twice
- [ ] start, resolve, reject — never settles

Check answer

2. Do `.then` handlers ever run synchronously if the promise is already settled?

- [ ] No — they are always scheduled as microtasks, running after the current synchronous stack
- [ ] Yes, they run immediately when attached
- [ ] Only if attached before the promise settles
- [ ] Only inside an async function

Check answer

3. How do `Promise.all` and `Promise.allSettled` differ on failure?

- [ ] Both reject on the first failure

- `all` rejects on the first input rejection (fail-fast); `allSettled` never rejects and reports every outcome
- `allSettled` fails fast while `all` waits

Check answer

Previous
Callbacks

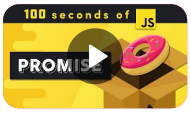
Next
Async / Await

LEARN MORE

ARTICLES & DOCS

- Using promises
  developer.mozilla.org
- Promise
  developer.mozilla.org
- Promise.prototype.then()
  developer.mozilla.org

VIDEOS



- JavaScript Promise in 100 Seconds
  Fireship



- JavaScript Promises In 10 Minutes
  Web Dev Simplified

Hand-picked references for this lesson.